Book Manager — Real-Time

A full-stack real-time book management application built with **React**, **Express**, and **Socket.IO**. Multiple users can add, edit, and delete books simultaneously — every change is broadcast to all connected clients instantly via WebSockets, keeping the UI in sync across tabs, browsers, and devices without manual refresh.

Table of Contents

- Project Overview
- Features
- Tech Stack
- Architecture
- Real-Time Event Flow
- Project Structure
- Running the Project Locally
- Accessing from Other Devices
- Testing Real-Time Updates
- WebSocket Connection Handling
- Deployment Plan
- Troubleshooting
- Monitoring and Post-Deployment Checks

Project Overview

Book Manager is a dashboard-style web application that demonstrates real-time communication between a server and multiple clients. When any user adds, edits, or deletes a book, every connected client receives the update immediately through WebSocket events — no polling, no page reload.

The backend uses an in-memory data store (pre-seeded with three sample books) and exposes a REST API for CRUD operations. Socket.IO handles the real-time broadcast layer. The frontend is a responsive React SPA styled with Tailwind CSS.

Features

- **Add books** — Submit a title and author; the new book appears on all connected clients instantly.
- **Edit books inline** — Click the edit button on any book card to modify its title or author in place.
- **Delete books** — Remove a book from the collection; deletion is broadcast in real time.
- **Soft locking** — When a user starts editing a book, other users see a lock indicator with the editor's name. Prevents conflicting edits.
- **Lock auto-expiry** — Locks automatically expire after 15 seconds of inactivity to prevent stale locks from disconnected or idle users.
- **Real-time activity feed** — A collapsible timeline showing recent add, update, and delete events with timestamps. Capped at 20 entries.

- **Connected users counter** — A live count of currently connected clients displayed in the header.
- **Live/Offline status indicator** — A pulsing badge showing whether the WebSocket connection is active.
- **Responsive card gallery** — Book cards with gradient covers displayed in a 2/3/4 column grid (mobile/tablet/desktop).
- **Skeleton loading states** — Placeholder cards shown while initial data loads.
- **Cross-device access** — API URLs use `window.location.hostname` so the app works on any device on the same network.

Tech Stack

Frontend

Technology	Purpose
React 18	UI components and state management
Vite 6	Build tool and dev server
Tailwind CSS 4	Utility-first styling via `@tailwindcss/vite` plugin
Axios	HTTP client for REST API calls
Socket.IO Client 4	WebSocket connection to the backend

Backend

Technology	Purpose
Node.js	Runtime environment
Express 4	HTTP server and REST API routing
Socket.IO 4	WebSocket server for real-time events
CORS	Cross-origin request handling
Nodemon	Auto-restart during development

Architecture

The application follows a modular client-server architecture. The frontend communicates with the backend through two channels:

1. **REST API (HTTP)** — For CRUD operations (GET, POST, PUT, DELETE on `/books`).
2. **WebSocket (Socket.IO)** — For real-time event broadcasting and soft-lock coordination.

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Client (React)                              │
│                                                                      │
│  ┌──────────┐  ┌──────────┐  ┌──────────────┐  ┌────────────────┐  │
```

```
│   │ AddBook  │   │ BookList │   │ActivityFeed│   │   App.jsx    │   │
│   └──────────┘   └──────────┘   └────────────┘   └──────────────┘   │
│        │             │              │                  │            │
│        └─────────────┴──────────────┴──────────────────┘            │
│                      │              │                               │
│              axios (REST)    socket.io-client                       │
└──────────────────────┼──────────────┼───────────────────────────────┘
                       │              │
                       ▼              ▼
┌──────────────────────┼──────────────┼───────────────────────────────┐
│                      │   Express    │   Server (Node.js)            │
│                      │              │                               │
│               ┌──────┴──────┐  ┌────┴────────────┐                  │
│               │  REST API   │  │  Socket.IO      │                  │
│               │  /books     │  │  Events         │                  │
│               └─────────────┘  └─────────────────┘                  │
│                      │              │                               │
│               ┌──────┴──────┐  ┌────┴────────────┐                  │
│               │ Controllers │  │SocketHandler    │                  │
│               └─────────────┘  └─────────────────┘                  │
│                      │                                              │
│               ┌──────┴──────┐                                       │
│               │  Services   │  (in-memory data store)               │
│               └─────────────┘                                       │
└─────────────────────────────────────────────────────────────────────┘
```

Request flow:

```
Client → REST (POST/PUT/DELETE) → Express Route → Controller → Service
(mutate data)
                                                        │
                                                        ├──
io.emit('bookAdded' / 'bookUpdated' / 'bookDeleted')
                                                        └──
io.emit('activity', { type, title, timestamp })
                                                                  │
                                                                  ▼
                                                        All connected
clients update in real time
```

Real-Time Event Flow

When a book is added

1. User fills in the form in `AddBook` and submits.
2. `AddBook` sends a `POST /books` request via Axios.
3. The backend controller validates the input, creates the book via `bookService.createBook()`, then emits two Socket.IO events to **all** clients:

    - `bookAdded` — contains the new book object.
    - `activity` — contains `{ type: 'BOOK_ADDED', title, timestamp }`.
4. Every client's `App.jsx` receives `bookAdded` and appends the book to state.
5. Every client's `ActivityFeed` receives `activity` and prepends the event to the feed.

When a book is updated

1. User clicks Edit on a book card in `BookList`.
2. `BookList` emits a `startEditing` Socket.IO event with `{ bookId }`.
3. The server records the lock in `editLocks` and broadcasts `bookLocked` to other clients.
4. User modifies fields and clicks Save.
5. `BookList` sends a `PUT /books/:id` request via Axios.
6. The backend controller validates, updates via `bookService.updateBook()`, and emits `bookUpdated` + `activity`.
7. `BookList` emits `stopEditing` to release the lock; server broadcasts `bookUnlocked`.
8. All clients update the book in their list and see the activity event.

When a book is deleted

1. User clicks Delete while in edit mode.
2. `BookList` sends a `DELETE /books/:id` request via Axios.
3. The backend controller removes the book via `bookService.deleteBook()` and emits `bookDeleted` + `activity`.
4. All clients remove the book from their list.

Socket.IO Events Reference

Event	Direction	Payload	Description
`bookAdded`	Server → All clients	`{ id, title, author }`	A new book was created
`bookUpdated`	Server → All clients	`{ id, title, author }`	A book was modified
`bookDeleted`	Server → All clients	`{ id }`	A book was removed
`activity`	Server → All clients	`{ type, title, timestamp }`	Activity feed event
`usersOnline`	Server → All clients	`number`	Updated connected user count
`startEditing`	Client → Server	`{ bookId, userName? }`	Client begins editing a book
`stopEditing`	Client → Server	`{ bookId }`	Client finishes editing

Event	Direction	Payload	Description
`bookLocked`	Server → Other clients	`{ bookId, userName }`	A book is being edited
`bookUnlocked`	Server → Other clients	`{ bookId }`	A book is no longer being edited
`lockExpired`	Server → Editing client	`{ bookId }`	Lock auto-expired after timeout
`currentLocks`	Server → New client	`{ [bookId]: { socketId, userName } }`	Existing locks on connect
`requestUsersOnline`	Client → Server	*(none)*	Request current user count

REST API Reference

Method	Path	Body	Description
GET	`/books`	—	List all books
POST	`/books`	`{ title, author }`	Create a book
PUT	`/books/:id`	`{ title, author }`	Update a book
DELETE	`/books/:id`	—	Delete a book

Project Structure

```
book-realtime-app/
├── backend/
│   ├── server.js                    # Entry point: Express + Socket.IO
setup
│   ├── package.json                 # Backend dependencies and scripts
│   ├── controllers/
│   │   └── bookController.js        # HTTP handlers (GET, POST, PUT,
DELETE)
│   ├── routes/
│   │   └── bookRoutes.js            # Express router mapping verbs to
controllers
│   ├── services/
│   │   └── bookService.js           # In-memory data store with CRUD
functions
│   └── sockets/
│       └── socketHandler.js         # Socket.IO lifecycle, user count,
soft locks
│
├── frontend/
│   ├── index.html                   # HTML shell with Inter font and
Tailwind body classes
```

```
│   ├── package.json                # Frontend dependencies and scripts
│   ├── vite.config.js              # Vite config with React and Tailwind
plugins
│   └── src/
│       ├── main.jsx                # React entry point
│       ├── App.jsx                 # Root layout, state management,
socket listeners
│       ├── AddBook.jsx             # Form component for adding books
│       ├── BookList.jsx            # Card gallery with edit/delete, lock
indicators
│       ├── ActivityFeed.jsx        # Collapsible real-time activity
timeline
│       ├── socket.js               # Singleton Socket.IO client instance
│       └── index.css               # Tailwind import and custom
animations
│
└── README.md
```

Backend layers:

- **Routes** — Thin mapping of HTTP methods to controller functions.
- **Controllers** — Validate requests, call services, emit Socket.IO events, send responses.
- **Services** — Pure data-access layer with no knowledge of HTTP or sockets.
- **Sockets** — Manages connection lifecycle, user count, and soft-lock state.

Running the Project Locally

Prerequisites

- **Node.js** 18 or later
- **npm** (comes with Node.js)

> **macOS note:** Port 5000 is reserved by AirPlay Receiver on macOS Monterey and later. This project uses port **5001** to avoid that conflict.

1. Clone the repository

```
git clone <repository-url>
cd book-realtime-app
```

2. Install and start the backend

```
cd backend
npm install
npm run dev
```

The backend starts on **port 5001** at `http://localhost:5001`.

3. Install and start the frontend

In a separate terminal:

```
cd frontend
npm install
npm run dev
```

Vite starts the dev server (typically at `http://localhost:5173`). Open this URL in your browser.

Accessing from Other Devices

To access the app from a phone or another computer on the same network:

1. Start the frontend with host binding:

   ```
   npm run dev -- --host
   ```

2. Find your machine's local IP:

   ```
   # macOS
   ipconfig getifaddr en0

   # Linux
   hostname -I
   ```

3. On the other device, open `http://<your-ip>:5173` in a browser.

The frontend automatically uses `window.location.hostname` for API and socket URLs, so it connects to the correct backend without code changes.

Testing Real-Time Updates

1. Open the app in **two or more browser tabs** (or on different devices).
2. **Add a book** in one tab — it appears in all other tabs instantly.
3. **Edit a book** in one tab — other tabs show a lock indicator with the editor's name on hover.
4. **Save the edit** — the updated title/author appears everywhere; the lock is released.
5. **Delete a book** — it disappears from all tabs.
6. **Watch the activity feed** — every add, update, and delete event appears in the collapsible timeline.
7. **Check the user count** — the "X online" badge in the header updates as tabs are opened or closed.
8. **Test lock expiry** — start editing a book, wait 15 seconds without saving, and the lock releases automatically.

WebSocket Connection Handling

Connection

- The Socket.IO client is created as a **singleton** in `socket.js` and shared across all components.
- On connect, the server increments the user counter and broadcasts `usersOnline` to all clients.
- The server sends `currentLocks` to the newly connected client so it can display any active edit locks.

Disconnection

- On disconnect, the server decrements the user counter and broadcasts the updated count.
- All edit locks held by the disconnected socket are released, and `bookUnlocked` is broadcast.
- Lock expiry timers for that socket are cleared.

Reconnection

- The client is configured with automatic reconnection:
  - Up to **5 reconnection attempts**
  - **1-second delay** between attempts
- Connection state is reflected in the UI as a **Live** (green) or **Offline** (red) badge.
- On reconnect, the client re-requests the user count via a `requestUsersOnline` event to handle race conditions where the `usersOnline` event fires before React listeners are registered.

Error Handling

- Connection errors are logged to the browser console.
- If all reconnection attempts fail, a `reconnect_failed` event is logged.
- If the initial book fetch fails, an error banner is displayed in the UI.

Deployment Plan

Backend (e.g., Render, Railway, Fly.io)

1. Set the `PORT` environment variable (the server reads `process.env.PORT` with a fallback of 5001).
2. Set the start command to `npm start` (runs `node server.js`).
3. Update the Socket.IO CORS `origin` in `server.js` to your frontend's production URL instead of `'*'`.
4. For persistent data, replace the in-memory store in `bookService.js` with a database (e.g., PostgreSQL, MongoDB).

Frontend (e.g., Vercel, Netlify, Cloudflare Pages)

1. Build the production bundle:

```
cd frontend
npm run build
```

2. Deploy the `dist/` folder.
3. Update the API URL in `socket.js` and the `API_URL` constants to point to the deployed backend URL. Consider using an environment variable:

```
const API_URL = import.meta.env.VITE_API_URL ||
`http://${window.location.hostname}:5001`;
```

Environment Variables (production)

Variable	Used By	Description
`PORT`	Backend	Server port (default: 5001)
`VITE_API_URL`	Frontend (if added)	Backend URL for API and socket

Troubleshooting

Problem	Cause	Solution
"Offline" badge on first load	Socket connects before React listeners register (common in Safari)	Already handled — the app re-requests state after listener setup. Hard refresh if it persists.
"Could not load books" error	Backend is not running or wrong port	Start the backend with `npm run dev` and verify it's on port 5001.
Socket connection error in console	Backend not reachable	Check that both servers are running. If on another device, use `--host` and the correct IP.
Port 5001 already in use	Another process on the port	Kill it: `lsof -ti:5001 | xargs kill` or change the port in `server.js`.
Edits not saving / lock stuck	Lock expired (15s timeout)	The lock auto-releases. Start editing again to re-acquire.
Changes not appearing on phone	Phone can't reach localhost	Start frontend with `npm run dev -- --host` and use the machine's LAN IP.
Tailwind styles not applied	Missing plugin	Run `npm install` in frontend; verify `vite.config.js` includes `tailwindcss()`.

Monitoring and Post-Deployment Checks

After deploying the application, several checks are performed to ensure that the real-time communication system works reliably.

Runtime Monitoring

The backend logs key Socket.IO lifecycle events:

- client connections
- client disconnections
- reconnection attempts
- connection errors

These logs help detect issues with WebSocket connectivity or unexpected client drops.

Example events monitored:

- `connect`
- `disconnect`
- `connect_error`
- `reconnect_attempt`

Deployment Platform Logs

The application is deployed on Railway, which provides runtime logs for the Node.js server. These logs allow monitoring of:

- server startup
- WebSocket connections
- API request errors
- unexpected crashes

Post-Deployment Verification

After deployment, the following checks were performed:

1. Open the application in multiple browser tabs.
2. Add or edit a book in one tab.
3. Confirm that updates appear instantly on other tabs.
4. Verify that the connected users counter updates when tabs are opened or closed.
5. Confirm that WebSocket reconnection restores the live connection after temporary network interruption.